

Repository Plan for `mini-llm-lab`

Project design document

September 26, 2026

1 Purpose of the repository

This repository is meant to be a learning platform for building a small modern **large language model (LLM)** from the ground up while keeping the mathematics, the matrices, and the numerical flow visible.

The central idea is:

Every important part of the model should be inspectable as explicit matrices, explicit tensor operations, and explicit backward derivatives.

The project is therefore *not* primarily about obtaining the strongest model possible. Instead, it is about understanding:

- how text becomes tokens,
- how tokens become vectors,
- how **Transformer** blocks propagate information,
- how **attention** works,
- how residual connections work,
- how the next-token prediction loss is formed,
- how gradients move backward through the model, and
- how optimization updates the matrices.

2 Main design philosophy

The repository follows the following rules.

Rule 1: explicit numerical objects

All trainable weights should appear as visible matrices, usually through a small `Parameter` class with:

`data,` `grad.`

Rule 2: explicit forward pass

Each module should compute its forward operation with explicit tensor contractions, for example:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

rather than hiding the operation behind a high-level neural-network framework layer.

Rule 3: explicit backward pass

Each module should also contain its own hand-derived backward pass. For example, if

$$Y = XW,$$

then the backward formulas are:

$$\frac{\partial L}{\partial W} = X^\top \frac{\partial L}{\partial Y}, \quad \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} W^\top.$$

Rule 4: CPU first, GPU second

The same code should run with:

- **NumPy** for a clear **central processing unit (CPU)** reference implementation,
- **CuPy** for **graphics processing unit (GPU)** acceleration.

Rule 5: verify before scaling

Every operator should be tested numerically before being integrated into a larger model. In practice this means:

- directional derivative checks,
- finite-difference tests,
- shape checks,
- causality checks for attention,
- and small overfitting experiments.

3 Acronyms and terms

Term	Meaning
LLM	Large language model. A neural network trained to predict the next token in text.
CPU	Central processing unit. The ordinary processor of the computer.
GPU	Graphics processing unit. A highly parallel processor used for accelerated tensor operations.
BPE	Byte-pair encoding. A tokenizer method that merges frequent token pairs to build a vocabulary.
LM	Language model. In this repository, usually a decoder-only next-token prediction model.
GQA	Grouped-query attention. A variant of attention where several query heads share the same key and value heads.
RoPE	Rotary positional embedding, more precisely a rotary positional transformation applied to query and key vectors.
RMSNorm	Root-mean-square normalization. A normalization layer using the root mean square of the input vector.

FFN	Feed-forward network. In this project, the position-wise non-attention block inside a Transformer block.
SwiGLU	Swish Gated Linear Unit. A feed-forward structure using a SiLU (sigmoid linear unit) gate.
MoE	Mixture of experts. A sparse feed-forward structure in which a router activates only some experts for each token.
KV cache	Key-value cache. Saved keys and values used during autoregressive generation to avoid recomputing the whole prefix.
CE loss	Cross-entropy loss. The loss used for next-token prediction.
AdamW	An optimization method based on Adam with decoupled weight decay.
SFT	Supervised fine-tuning. A later-stage training step used to teach instruction-following behavior.

4 Repository structure

The repository is planned to evolve toward the following structure:

```

mini_llm_lab/
|-- mini_llm/
|   |-- backend.py
|   |-- config.py
|   |-- init.py
|   |-- parameter.py
|   |-- ops/
|   |   |-- linear.py
|   |   |-- embedding.py
|   |   |-- rmsnorm.py
|   |   |-- rope.py
|   |   |-- attention.py
|   |   |-- swiglu.py
|   |   |-- loss.py
|   |-- optim/
|   |   |-- adamw.py
|   |   |-- grad_clip.py
|   |   |-- schedule.py
|   |-- diagnostics/
|   |   |-- gradcheck.py
|   |-- blocks/
|   |   |-- transformer_block.py
|   |   |-- moe.py
|   |-- model/
|   |   |-- decoder_lm.py
|   |   |-- inference.py
|   |-- data/
|   |   |-- tokenizer.py
|   |   |-- preprocess.py
|   |   |-- dataloader.py
|-- tests/
|-- examples/

```

`'-- docs/`

5 Planned milestones

Milestone 1: numerical primitives (already started)

Implemented or being implemented:

- backend selection (NumPy / CuPy),
- parameter objects,
- initialization,
- linear layers,
- embedding lookup,
- RMSNorm,
- RoPE,
- grouped-query causal attention,
- SwiGLU,
- cross-entropy loss,
- gradient clipping,
- AdamW,
- numerical derivative checks.

Milestone 2: dense Transformer block

Build the standard dense Transformer block:

$$X \rightarrow \text{RMSNorm} \rightarrow \text{Attention} \rightarrow +X \rightarrow \text{RMSNorm} \rightarrow \text{SwiGLU} \rightarrow +X.$$

Milestone 3: dense decoder language model

Stack multiple Transformer blocks, tie the input embedding matrix to the output language-model head, and train a small decoder-only language model on a tiny corpus.

Milestone 4: sparse feed-forward replacement

Replace the dense feed-forward network by a **mixture of experts (MoE)** block:

$$\text{Dense FFN} \longrightarrow \text{Router} + \text{Experts}.$$

Milestone 5: data pipeline

Build the tokenizer and dataset pipeline:

- raw documents,
- training/validation split,
- tokenizer training,
- token packing,
- binary token shards.

Milestone 6: full training loop

Create:

- checkpoints,
- logging,
- learning-rate schedules,
- validation,
- and loss/gradient statistics.

Milestone 7: inference

Add:

- autoregressive decoding,
- sampling,
- key-value cache,
- prompt processing.

Milestone 8: research experiments

Compare:

- dense vs. MoE,
- AdamW vs. Muon,
- data mixtures,
- and routing behavior.

6 Model overview

The full model will be a decoder-only Transformer for next-token prediction.

6.1 Input text and tokenization

A raw text sequence such as

"The fluid velocity increases near the wall."

is converted to discrete token identifiers using a tokenizer such as **byte-pair encoding (BPE)**:

$$[t_0, t_1, \dots, t_{T-1}].$$

6.2 Embedding

Each token identifier is converted into a vector through an embedding matrix

$$E \in \mathbb{R}^{V \times d_{\text{model}}},$$

where:

- V is the vocabulary size,
- d_{model} is the model dimension.

If X_0 denotes the embedded token sequence, then

$$X_0 \in \mathbb{R}^{B \times T \times d_{\text{model}}},$$

where B is the batch size and T is the sequence length.

6.3 How vectors, “nodes”, and weight matrices should be interpreted

It is useful to connect the usual neural-network picture of nodes and edges to the actual arrays stored in the repository. A token representation

$$x = (x_1, \dots, x_{d_{\text{model}}})$$

can be pictured as a row of d_{model} scalar nodes. A learned linear map does not store explicit graph edges. Instead, all connection weights are stored in one matrix. For

$$y = xW, \quad W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}},$$

the scalar output node y_j is

$$y_j = \sum_{i=1}^{d_{\text{in}}} x_i W_{ij}.$$

Thus W_{ij} is exactly the weight of the connection from input feature i to output feature j . In the code, opening `W.data` exposes this complete matrix directly.

There are two conceptually different kinds of coupling in a Transformer:

- **Feature coupling:** matrices such as W_Q , W_K , W_V , W_O , W_{gate} , and W_{up} mix coordinates inside one token representation.
- **Token-to-token coupling:** the attention matrix P mixes information between different sequence positions. It is not a learned parameter; it is recomputed from the current queries and keys for every input.

The feed-forward network therefore acts independently at each token position, whereas attention is what couples different token positions.

6.4 Transformer block: detailed structure

A Transformer block is the repeated computational unit of the model. In a modern decoder-only language model, every token position carries a vector of length d_{model} . For a batch of B sequences of length T , the input to one block is therefore

$$X \in \mathbb{R}^{B \times T \times d_{\text{model}}}.$$

The block preserves this outer shape. Internally, however, the vector is split into multiple attention heads and is also temporarily expanded inside the feed-forward or mixture-of-experts sublayer.

The block has two main residual branches:

$$\boxed{\text{attention branch}} \quad \text{and} \quad \boxed{\text{feed-forward / MoE branch}}.$$

A residual branch computes a transformation $F(X)$ and adds it back to the original signal:

$$Y = X + F(X).$$

This means that the block can learn a correction to the existing representation instead of replacing it completely.

6.4.1 First normalization: RMSNorm

The first operation is root-mean-square normalization (RMSNorm). For one token vector $x \in \mathbb{R}^{d_{\text{model}}}$,

$$r(x) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} x_j^2 + \epsilon},$$

$$\hat{x}_j = \frac{x_j}{r(x)}, \quad z_j = \gamma_j \hat{x}_j.$$

The learned parameter

$$\gamma \in \mathbb{R}^{d_{\text{model}}}$$

rescales each component after normalization. The tensor shape remains $B \times T \times d_{\text{model}}$.

6.4.2 Query, key and value projections

For the normalized tensor Z , three learned linear maps create the query, key, and value tensors:

$$Q_{\text{flat}} = ZW_Q, \quad K_{\text{flat}} = ZW_K, \quad V_{\text{flat}} = ZW_V.$$

For grouped-query attention (GQA), the matrix dimensions are

$$W_Q \in \mathbb{R}^{d_{\text{model}} \times (H_Q d_h)},$$

$$W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times (H_{KV} d_h)},$$

where

- H_Q is the number of query heads,
- H_{KV} is the number of key/value heads,
- d_h is the dimension of one attention head.

We choose

$$H_Q d_h = d_{\text{model}}.$$

For the planned small model,

$$d_{\text{model}} = 384, \quad H_Q = 6, \quad H_{KV} = 2, \quad d_h = 64.$$

Hence

$$W_Q \in \mathbb{R}^{384 \times 384}, \quad W_K, W_V \in \mathbb{R}^{384 \times 128}.$$

After reshaping,

$$Q \in \mathbb{R}^{B \times T \times 6 \times 64}, \\ K, V \in \mathbb{R}^{B \times T \times 2 \times 64}.$$

6.4.3 What is an attention head?

An attention head is not a separate neuron or a separate network. It is one slice of the projected feature space. For query head h , token t has a vector

$$q_{t,h} \in \mathbb{R}^{d_h}.$$

That vector is compared with the key vectors of earlier tokens to determine how much information should be read from their value vectors.

With GQA, several query heads share one key/value head. In the proposed configuration the mapping is

$$(Q_0, Q_1, Q_2) \longrightarrow (K_0, V_0), \\ (Q_3, Q_4, Q_5) \longrightarrow (K_1, V_1).$$

This reduces the size of the key-value cache during generation while preserving multiple query heads.

6.4.4 Rotary positional embedding (RoPE)

Attention by itself does not know token order. Rotary positional embedding (RoPE) encodes position by rotating pairs of coordinates of the query and key vectors. For one coordinate pair,

$$\begin{bmatrix} q'_{2i} \\ q'_{2i+1} \end{bmatrix} = \begin{bmatrix} \cos \theta_{t,i} & -\sin \theta_{t,i} \\ \sin \theta_{t,i} & \cos \theta_{t,i} \end{bmatrix} \begin{bmatrix} q_{2i} \\ q_{2i+1} \end{bmatrix}.$$

The same transformation is applied to K . In the implementation this rotation is performed directly on coordinate pairs rather than by constructing a large block-diagonal matrix, although a debug function can materialize that matrix for inspection.

6.4.5 Causal attention scores

For one query head, the raw attention score between query token t and key token s is

$$S_{t,s} = \frac{q_t^\top k_s}{\sqrt{d_h}}.$$

The factor $1/\sqrt{d_h}$ prevents the dot products from becoming excessively large as the head dimension grows.

Because this is an autoregressive decoder, token t must not see future token $s > t$. We therefore add a causal mask M :

$$M_{t,s} = \begin{cases} 0, & s \leq t, \\ -\infty, & s > t. \end{cases}$$

The normalized attention weights are

$$P_{t,s} = \text{softmax}_s(S_{t,s} + M_{t,s}).$$

Thus every row of P sums to one, but only over the current and previous tokens.

For all heads and batches the score tensor has shape

$$S, P \in \mathbb{R}^{B \times H_Q \times T \times T}.$$

This $T \times T$ object is the main reason ordinary full attention has quadratic memory and compute scaling with sequence length.

6.4.6 Reading information through the value vectors

The attention probability matrix is used to form a weighted average of the value vectors:

$$C_{t,h} = \sum_{s \leq t} P_{h,t,s} v_{s,h}.$$

After all query heads are concatenated,

$$C_{\text{flat}} \in \mathbb{R}^{B \times T \times (H_Q d_h)} = \mathbb{R}^{B \times T \times d_{\text{model}}}.$$

The attention output projection is

$$A = C_{\text{flat}} W_O, \quad W_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}.$$

The first residual update is therefore

$$X' = X + A.$$

6.4.7 Second normalization

The second branch begins with another RMSNorm:

$$Z_2 = \text{RMSNorm}(X').$$

From here the model can use either a dense feed-forward network or a sparse mixture-of-experts feed-forward network.

6.4.8 Dense SwiGLU feed-forward path

In the dense baseline every token passes through the same feed-forward matrices:

$$G = Z_2 W_{\text{gate}}, \quad U = Z_2 W_{\text{up}},$$

$$H = \text{SiLU}(G) \odot U,$$

$$F = H W_{\text{down}}.$$

Here $\odot$ denotes elementwise multiplication and SiLU denotes the sigmoid linear unit activation,

$$\text{SiLU}(x) = x \sigma(x).$$

For feed-forward width d_{ff} ,

$$W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}},$$

$$W_{\text{down}} \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}.$$

For the proposed model, $d_{\text{ff}} = 1024$. The second residual update is

$$Y = X' + F.$$

6.4.9 Sparse mixture-of-experts replacement

In the MoE version, the attention branch is unchanged. Only the dense feed-forward path is replaced. Instead of using one SwiGLU network for every token, the model contains several SwiGLU experts:

$$E_0, E_1, \dots, E_{N_E-1}.$$

Each expert has its own matrices

$$W_{\text{gate}}^{(e)}, \quad W_{\text{up}}^{(e)}, \quad W_{\text{down}}^{(e)}.$$

A router chooses which experts process each token. The router is simply another learned matrix

$$W_R \in \mathbb{R}^{d_{\text{model}} \times N_E}.$$

For one normalized token vector z ,

$$r = zW_R \in \mathbb{R}^{N_E}$$

contains one routing logit per expert. A softmax gives routing probabilities

$$p_e = \frac{\exp(r_e)}{\sum_j \exp(r_j)}.$$

For the planned model,

$$N_E = 6, \quad k = 2,$$

so only the two highest-probability experts are selected for a token. For example, if

$$p = [0.03, 0.12, 0.51, 0.07, 0.23, 0.04],$$

then experts 2 and 4 are selected. Their selected probabilities are renormalized:

$$\tilde{p}_2 = \frac{0.51}{0.51 + 0.23}, \quad \tilde{p}_4 = \frac{0.23}{0.51 + 0.23}.$$

The sparse feed-forward output becomes

$$F_{\text{MoE}}(z) = \tilde{p}_2 E_2(z) + \tilde{p}_4 E_4(z).$$

The other experts do not evaluate this token. This is the source of sparse conditional computation: the model owns many parameters, but only a subset is active for a given token.

The second residual update becomes

$$Y = X' + F_{\text{MoE}}(Z_2).$$

6.4.10 Optional shared expert

A later variant will add a shared expert that is always active:

$$F_{\text{MoE}}(z) = E_{\text{shared}}(z) + \sum_{e \in \text{TopK}(p)} \tilde{p}_e E_e(z).$$

The shared expert can represent transformations useful to most tokens, while the routed experts are free to specialize.

6.4.11 How the top-k selection is differentiated

The expert indices selected by top- k are discrete. During the local backward pass, the selected expert identities are treated as fixed and gradients propagate through:

- the selected expert outputs,
- their selected routing weights,
- and therefore the router logits that produced those weights.

At the exact point where two experts exchange rank, the top- k selection is non-smooth. This is important when performing finite-difference checks: a numerical perturbation that changes the selected expert set can cross a genuine routing boundary.

6.4.12 Why load balancing is required

Without additional control, a router can collapse and send most tokens to only a few experts. We will therefore record for each expert:

$$N_e = \text{number of routed tokens},$$

as well as routing probabilities and entropy. The first MoE implementation will use a conventional auxiliary balancing objective; a later version can test bias-based or auxiliary-loss-free balancing. The point of the repository is to make these choices measurable rather than hidden.

6.4.13 Dense and MoE blocks have the same external shape

Importantly, both variants map

$$\mathbb{R}^{B \times T \times d_{\text{model}}} \longrightarrow \mathbb{R}^{B \times T \times d_{\text{model}}}.$$

Therefore the rest of the decoder stack does not need to know whether the second branch is dense or sparse. This allows the repository to implement both versions behind a common block interface and compare them directly.

6.4.14 Tensor-shape trace for the planned model

For $d_{\text{model}} = 384$, $H_Q = 6$, $H_{KV} = 2$, $d_h = 64$, $d_{\text{ff}} = 1024$, $N_E = 6$, and $k = 2$, the main tensors inside one block are:

Quantity	Shape	Role
Block input X	$B \times T \times 384$	Current representation of every token.
Normalized input Z_1	$B \times T \times 384$	Input to attention.
Queries Q	$B \times T \times 6 \times 64$	Six query heads per token.
Keys/values K, V	$B \times T \times 2 \times 64$	Two shared key/value heads.
Logical GQA-expanded K, V	$B \times T \times 6 \times 64$	Each key/value head serves three query heads.
Scores/probabilities S, P	$B \times 6 \times T \times T$	Causal token-to-token attention matrices.
Head context C	$B \times T \times 6 \times 64$	Weighted sums of value vectors.
Merged attention output	$B \times T \times 384$	Concatenated attention heads.
Dense SwiGLU G, U, H	$B \times T \times 1024$	Expanded per-token feature space.
Dense FFN output F	$B \times T \times 384$	Returned to the residual stream.
MoE router logits/probabilities	$B \times T \times 6$	One score/probability per expert.
Top- k expert indices	$B \times T \times 2$	Two selected experts per token.
MoE combined output	$B \times T \times 384$	Weighted sum of selected expert outputs.
Block output Y	$B \times T \times 384$	Same external shape as the input.

The first implementation will **not drop tokens because of expert capacity**. Every top- k assignment will be evaluated. This keeps routing transparent; expert capacity limits and token dropping can be studied later as separate systems-level optimizations.

6.5 Stacking

If there are L layers, the sequence passes through:

$$X_0 \rightarrow X_1 \rightarrow X_2 \rightarrow \cdots \rightarrow X_L.$$

6.6 Final normalization and logits

After the last block:

$$Z_{\text{final}} = \text{RMSNorm}(X_L).$$

The logits are then computed using the tied embedding matrix:

$$\text{logits} = Z_{\text{final}} E^\top.$$

6.7 Next-token prediction

For each position t , the model predicts the next token:

$$p(x_{t+1} \mid x_{\leq t}) = \text{softmax}(\text{logits}_t).$$

Training uses the mean cross-entropy loss over all positions.

7 Numerical conventions and initialization

The initial implementation uses:

- clipped normal initialization for most matrices,
- smaller standard deviation for residual-output matrices,
- $\gamma = 1$ for RMSNorm gains,

- AdamW optimization,
- global gradient clipping,
- warmup and cosine learning-rate decay.

The main reason is not fashion; it is numerical stability and interpretability. A transparent project is easier to debug when:

- activation scales are monitored,
- gradient scales are monitored,
- the optimizer is simple and explicit,
- and each matrix has a clear role.

8 How training will be validated

Before any serious training run, the repository should verify:

1. **Operator checks:** each layer passes local finite-difference checks.
2. **Block checks:** a full Transformer block passes directional derivative checks.
3. **Model checks:** the full dense model passes end-to-end gradient tests on a tiny example.
4. **Memorization checks:** the model can intentionally overfit a tiny corpus.
5. **Scaling checks:** only after the above are correct do we train on larger corpora.

9 MoE design planned for this repository

The first complete language model will be dense because it is easier to verify. Once that implementation can overfit a tiny corpus and passes end-to-end gradient checks, the dense SwiGLU branch will be replaced by a mixture-of-experts (MoE) branch while leaving the attention sublayer unchanged.

9.1 Initial MoE configuration

The initial target configuration is:

$$N_E = 6 \text{ routed experts,} \quad k = 2,$$

with one optional shared expert added in a later experiment. Each routed expert is itself a complete SwiGLU feed-forward network.

For every token, the router performs the following sequence:

$$z \in \mathbb{R}^{d_{\text{model}}} \xrightarrow{W_R} r \in \mathbb{R}^{N_E} \xrightarrow{\text{softmax}} p \xrightarrow{\text{top-}k} \{e_1, \dots, e_k\}.$$

The selected probabilities are renormalized and used as mixture coefficients. Consequently, a token can use different feed-forward parameters from another token in the same sequence and even from another position in the same sentence.

9.2 Token-to-expert dispatch

For a batch containing BT token representations, the implementation will first flatten the token dimensions:

$$Z \in \mathbb{R}^{(BT) \times d_{\text{model}}}.$$

The router then computes

$$R = ZW_R \in \mathbb{R}^{(BT) \times N_E}.$$

After top- k selection, each token has an integer expert-index vector and a floating-point routing-weight vector. Conceptually:

$$\text{token } i \longrightarrow [(e_{i,1}, \tilde{p}_{i,1}), \dots, (e_{i,k}, \tilde{p}_{i,k})].$$

Tokens assigned to expert e are gathered into a temporary expert batch, processed by that expert, multiplied by the corresponding routing weights, and scattered back into their original token locations. If a token is sent to two experts, two weighted contributions are accumulated at the same output location.

9.3 What will be inspected

The repository will explicitly expose:

- the router matrix W_R ,
- router logits for each token,
- routing probabilities,
- selected expert indices,
- number of tokens handled by each expert,
- routing entropy,
- gradient norms for each expert,
- and expert activation statistics by data domain.

Because dataset metadata will retain whether a document came from prose, code, mathematics, or synthetic educational data, we will eventually be able to measure quantities such as

$$P(\text{expert } e \mid \text{code}) \quad \text{and} \quad P(\text{expert } e \mid \text{mathematics}).$$

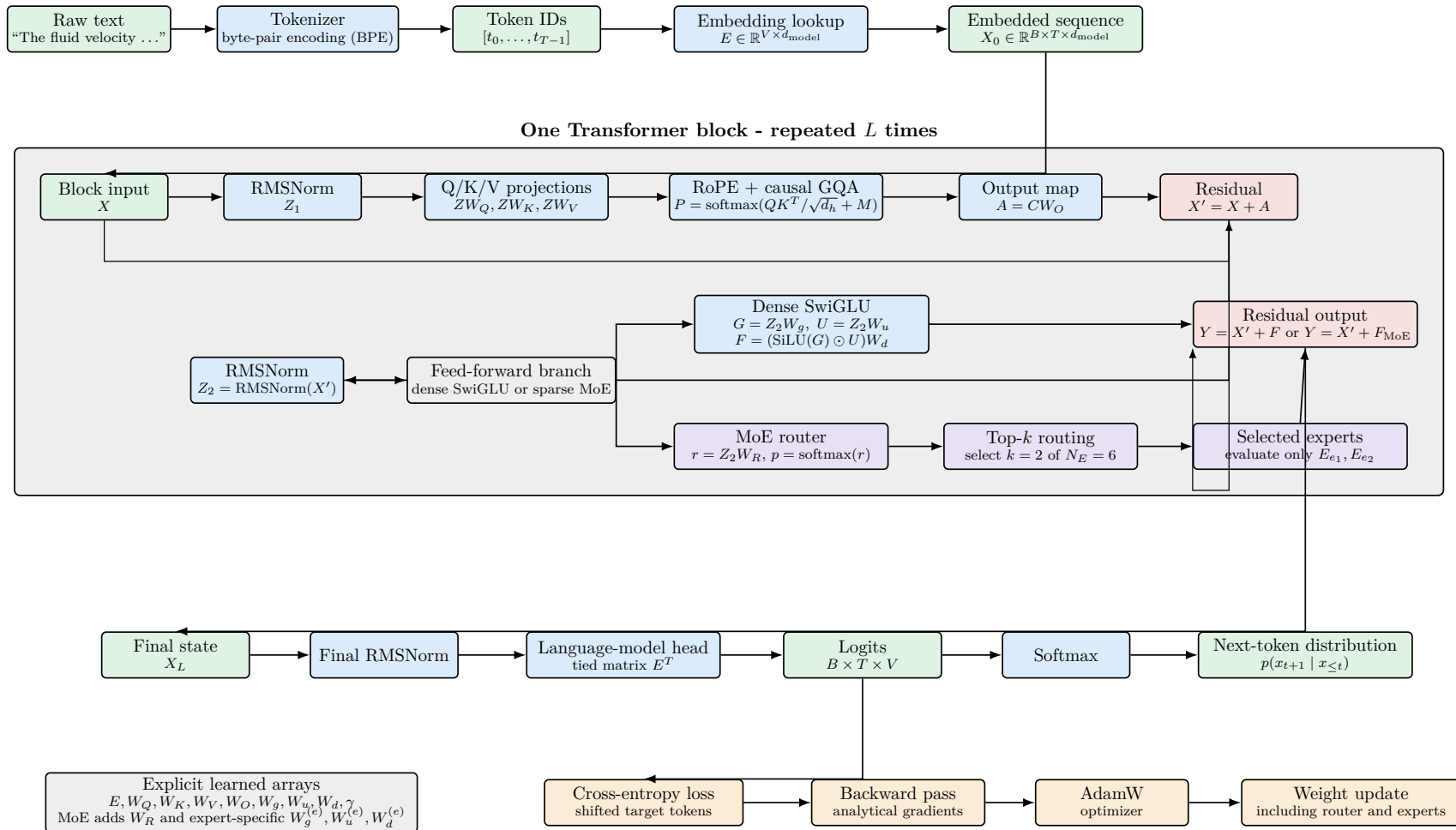
This provides a direct way to test whether the experts actually specialize.

9.4 Dense versus sparse comparison

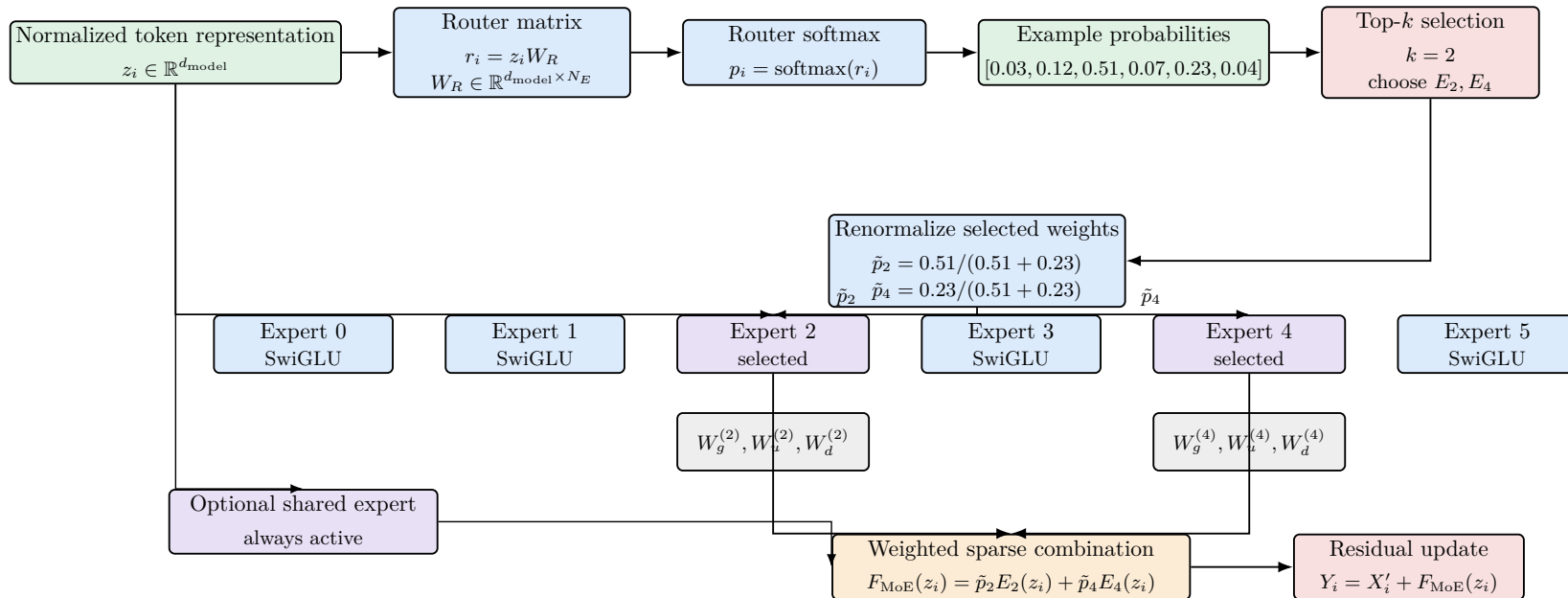
A central research comparison will hold the approximate active computation per token roughly fixed while increasing the total number of available parameters in the MoE model. We will then compare:

- training loss versus tokens seen,
- validation loss and perplexity,
- wall-clock throughput,
- active floating-point operations per token,
- router balance,
- and expert specialization.

Full model diagram: text to next-token prediction



Detailed mixture-of-experts routing diagram



Batch implementation: flatten $B \times T$ tokens $\rightarrow$ compute all router logits $\rightarrow$ gather token rows by expert
 $\rightarrow$ evaluate selected experts $\rightarrow$ multiply by routing weights $\rightarrow$ scatter-add contributions back to original token positions

Backward pass: selected expert indices are locally fixed; gradients flow through selected experts and selected routing weights.
 Top- k itself is non-smooth exactly when expert ranks exchange order.

10 Conclusion

This repository is intended to become a transparent experimental platform for understanding the full life of a language model:

raw text $\rightarrow$ tokens $\rightarrow$ embeddings $\rightarrow$ Transformer blocks $\rightarrow$ logits $\rightarrow$ cross-entropy loss $\rightarrow$ gradients $\rightarrow$ weight updates.

The project starts with dense explicit components, validates them carefully, and only then moves toward modern sparse mechanisms such as MoE routing and expert specialization.